

# Reviewer Pack — Additive Energy Inverse Proportional to Discrepancy of Sumse...

Entry c660a4274e70 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 17:52:40 UTC

## Additive Energy Inverse Proportional to Discrepancy of Sumset Communication Matrix

Entry ID: c660a4274e70 Verdict: INCONCLUSIVE Recorded: 2026-05-11 17:52:40 UTC

### 1. Conjecture

**Field A** (mathematical branch): Additive Combinatorics (Additive Energy) **Field B** (complexity object): Communication Complexity (Discrepancy of Communication Matrix)

#### Statement:

For a random subset  $A \subseteq \mathbb{Z}_n$ , the discrepancy of the communication matrix for  $f(x,y)=1$  iff  $x+y \in A$  is  $\Theta(1/E)$ , where  $E$  is the additive energy of  $A$ . Specifically,  $E = |\{(a,b,c,d) \in A^4 : a+b=c+d\}|$  and discrepancy is the minimum over all rectangle partitions of the absolute difference between the number of 1s and 0s in each rectangle.

#### Rationale (proposer's reasoning):

High additive energy implies structured additive quadruples, which may regularize the sumset function's communication matrix, reducing discrepancy. This bridges additive structure with communication complexity via dispersion methods.

**Taxonomy category:** DISPERSION\_DISCREPANCY (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 2ec996a07ad4efa2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.95       | SAFE             | UNCERTAIN        |

| Barrier       | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------------|---------|------------|------------------|------------------|
| ALGEBRIZATION | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON   | SAFE    | 0.95       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def additive_energy(A):
        n = len(A)
        E = 0
        for a in range(n):
            for b in range(a, n):
                for c in range(b, n):
                    for d in range(c, n):
                        if (A[a] + A[b]) % n == (A[c] + A[d]) % n:
                            E += 1
        return E

    def discrepancy(A):
        n = len(A)
        D = float('inf')
        for i in range(n):
            for j in range(i+1, n):
                for k in range(j+1, n):
                    for l in range(k+1, n):
                        count_ones = 0
                        for x in range(n):
                            for y in range(n):
                                if (A[x] + A[y]) % n == (A[i] + A[j]) % n and (x, y) not in [(i, j)]:
                                    count_ones += 1
                        D = min(D, abs(count_ones - (n*n - count_ones)))
        return D

    n = random.randint(5, 40)
    A = set(random.sample(range(n), random.randint(1, n//2)))

    E = additive_energy(A)
    D = discrepancy(A)

    if E == 0:
```

```

    return {
        "metric_name": "Discrepancy/E",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Additive energy is zero, discrepancy undefined."
    }

ratio = D / E
C = ratio * n**2

    return {
        "metric_name": "Discrepancy/E",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": abs(ratio - C) < 0.2,
        "counterexample": "" if abs(ratio - C) < 0.2 else f"Ratio {ratio} deviates by more than 20"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
        results.append(result)

    mean_ratio = sum(r['metric_value'] for r in results) / len(results)
    std_ratio = math.sqrt(sum((r['metric_value'] - mean_ratio)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(r['counterexample'] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{next(r['counterexample'] for r in results if r['conjecture_holds'])}\"")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_da260468.py", line 79, in <module>

```

    result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp\_sandbox/test\_da260468.py", line 50, in run\_trial

```

    E = additive_energy(A)
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_da260468.py", line 28, in additive_energy
    if (A[a] + A[b]) % n == (A[c] + A[d]) % n:
        ~^^^
```

TypeError: 'set' object is not subscriptable

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed due to TypeError in additive\_energy function (set subscripting). No data collected to assess conjecture. | next: Fix the additive\_energy function to handle sets properly (e.g., convert to list or use iteration)

## 11. Audit log (LLM calls)

**Total LLM calls:** 8

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 111722       |
| 2 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 62106        |
| 3 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 24220        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 21217        |
| 5 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 17170        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15142        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9652         |
| 8 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 17165        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 278393 ms total latency. Provider mix: {'ollama\_remote': 8}

(full prompt+response transcripts available in `research/audit/c660a4274e70.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c660a4274e70.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/c660a4274e70.tar.gz` (if generated)